PYTHON ACTIVITY-BASED LEARNING

PYTHON PROGRAMMING | CONSOLIDATED ABL REPORT

Student Name	N. Lakshmi Prasanna
Roll Number	26000963
Branch	CSM
Section	2
Institution	Dr. RVR NRI Institute of Technology
Course	Python Programming

SELECTED CASE STUDIES

1. Progressive Electricity Billing
2. Multi-Currency Travel Budget Analysis
3. Advanced Numeric Pattern Generator
4. Weekly Electricity Consumption Matrix
5. ATM Transaction Sequence Simulation
6. Warehouse Inventory Reconciliation
7. Next-Gen Loop Examination Seating Generator
8. University Examination Result Moderation
9. Smart Public Transport Fare Reconciliation
10. Water Distribution Fairness Analysis

Each case study includes the required documentation, a flowchart, Python code, test cases, and an output preview.

REPORT GUIDE

The ten case studies are presented once each, in the same order as the selected list. Each case has two consecutive pages: the first page contains the problem definition, objectives, assumptions, algorithm, and flowchart; the second page contains pseudocode, Python source code, test cases, output preview, and conclusion.

Flowchart symbol guide

Shape	Meaning
Oval	Start or Stop
Parallelogram	Input or Output
Rectangle	Processing / calculation
Diamond	Decision / condition

Note: Output preview images are formatted examples created from the sample input and expected output listed for each program. They are not photographs of a live Python terminal. Run each program in Python to capture an actual execution screenshot if your faculty specifically requires one.

CASE STUDY 1: Progressive Electricity Billing

1. Problem Statement

Design a Python program that calculates a household electricity bill from the number of units consumed. The program must apply progressive rates, meaning each rate applies only to the units that fall within its slab, rather than applying the highest rate to every unit.

2. Objectives

• Accept and validate the number of electricity units consumed.

• Calculate each applicable slab charge separately.

• Add the fixed service fee and display an itemised total.

• Handle invalid negative input safely.

3. Assumptions

• This is an educational sample tariff, not an official electricity tariff.

• First 100 units: ■1.50 per unit; next 100 units: ■2.50 per unit; units above 200: ■4.00 per unit.

• A fixed service charge of ■50 is added to every valid bill.

• Units are entered as whole numbers.

4. Algorithm

1. Start the program.

2. Read the units consumed.

3. If units are negative, show an error message.

4. If units are at most 100, multiply all units by ■1.50.

5. If units are between 101 and 200, charge the first 100 at ■1.50 and the remaining units at ■2.50.

6. If units exceed 200, charge the first 100 at ■1.50, the next 100 at ■2.50, and the rest at ■4.00.

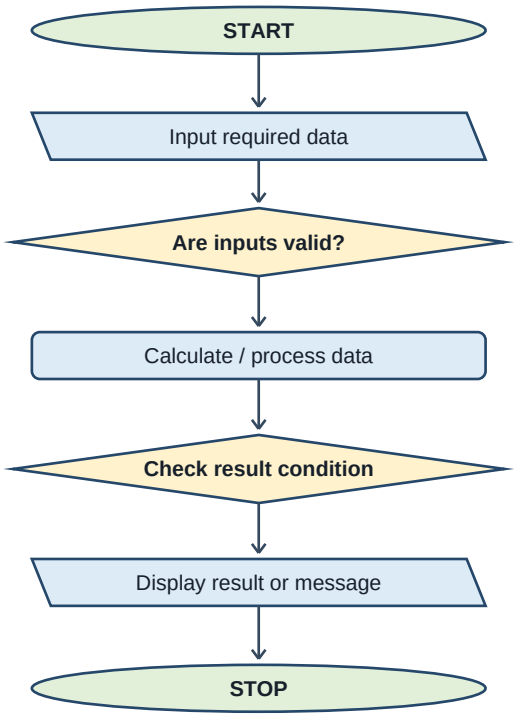
7. Add the ■50 service charge.

8. Display the energy charge and final bill, then stop.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 1: IMPLEMENTATION

6. Pseudocode

```
START
INPUT units
IF units < 0 THEN
    DISPLAY "Invalid units"
ELSE
    IF units <= 100 THEN
        energy = units * 1.50
    ELSE IF units <= 200 THEN
        energy = 100 * 1.50 + (units - 100) * 2.50
    ELSE
        energy = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00
    END IF
    total = energy + 50
    DISPLAY energy, total
END IF
STOP
```

7. Python Source Code

```
units = int(input("Enter units consumed: "))

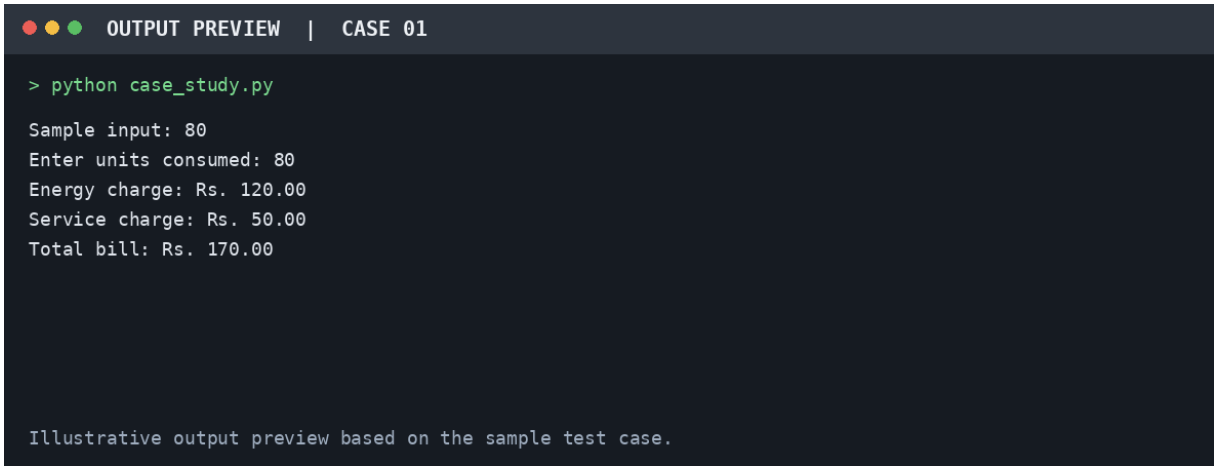
if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy_charge = units * 1.50
    elif units <= 200:
        energy_charge = 100 * 1.50 + (units - 100) * 2.50
    else:
        energy_charge = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00

    total_bill = energy_charge + 50
    print(f"Energy charge: Rs. {energy_charge:.2f}")
    print(f"Service charge: Rs. 50.00")
    print(f"Total bill: Rs. {total_bill:.2f}")
```

8. Test Cases

Test Input / Scenario	Expected Output
Units = 80	Energy charge ■120.00; Service charge ■50.00; Total bill ■170.00
Units = 150	Energy charge ■275.00; Service charge ■50.00; Total bill ■325.00
Units = 250	Energy charge ■550.00; Service charge ■50.00; Total bill ■600.00
Units = -4	Invalid units

9. Output Screenshot / Preview



10. Conclusion

The program calculates an electricity bill using progressive slabs, keeps the fixed fee separate, and rejects negative unit values. The sample tariff can be replaced with the tariff supplied by the electricity provider.

CASE STUDY 2: Multi-Currency Travel Budget Analysis

1. Problem Statement

Create a travel-budget calculator that converts a planned budget from Indian rupees into a selected foreign currency and checks whether the converted amount is enough to cover the estimated trip cost. The program should clearly report the available balance or the shortfall.

2. Objectives

• Accept the budget in INR, a currency code, and the estimated trip cost.

• Support USD, EUR, and GBP using a rate table.

• Validate the budget and expense values.

• Compare the converted budget with the planned cost and report the difference.

3. Assumptions

• Exchange rates are illustrative sample values, not live market rates.

• Sample rates: 1 INR = 0.012 USD, 0.011 EUR, and 0.0095 GBP.

• Budget and estimated trip cost cannot be negative.

• Currency codes are entered as USD, EUR, or GBP.

4. Algorithm

1. Start and read the INR budget, currency code, and trip cost.

2. Create a dictionary of the sample exchange rates.

3. If the budget or trip cost is negative, or the currency is unsupported, display an error.

4. Convert the INR budget using the selected rate.

5. Subtract the trip cost from the converted budget.

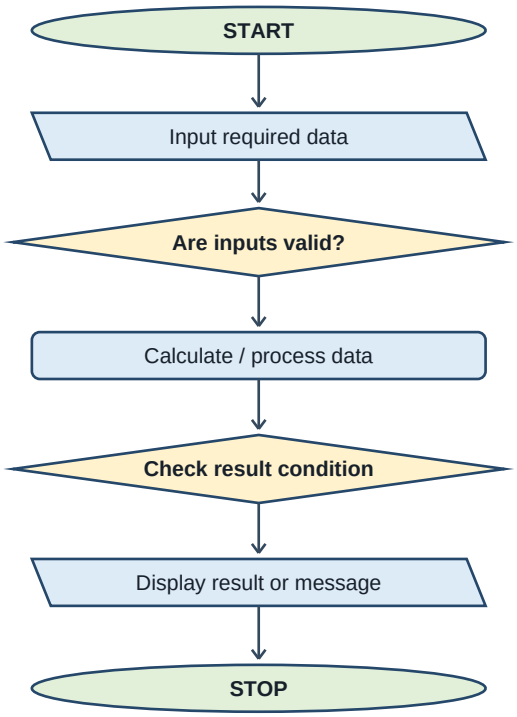
6. If the difference is non-negative, display the remaining budget; otherwise display the shortfall.

7. Stop.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 2: IMPLEMENTATION

6. Pseudocode

```
START
INPUT budget_inr, currency, trip_cost
rates = {USD: 0.012, EUR: 0.011, GBP: 0.0095}
IF budget_inr < 0 OR trip_cost < 0 OR currency not in rates THEN
    DISPLAY "Invalid input"
ELSE
    converted = budget_inr * rates[currency]
    difference = converted - trip_cost
    DISPLAY converted
    IF difference >= 0 THEN DISPLAY remaining budget
    ELSE DISPLAY shortfall = ABS(difference)
STOP
```

7. Python Source Code

```
budget_inr = float(input("Enter travel budget in INR: "))
currency = input("Choose currency (USD/EUR/GBP): ").upper()
trip_cost = float(input("Enter estimated trip cost: "))

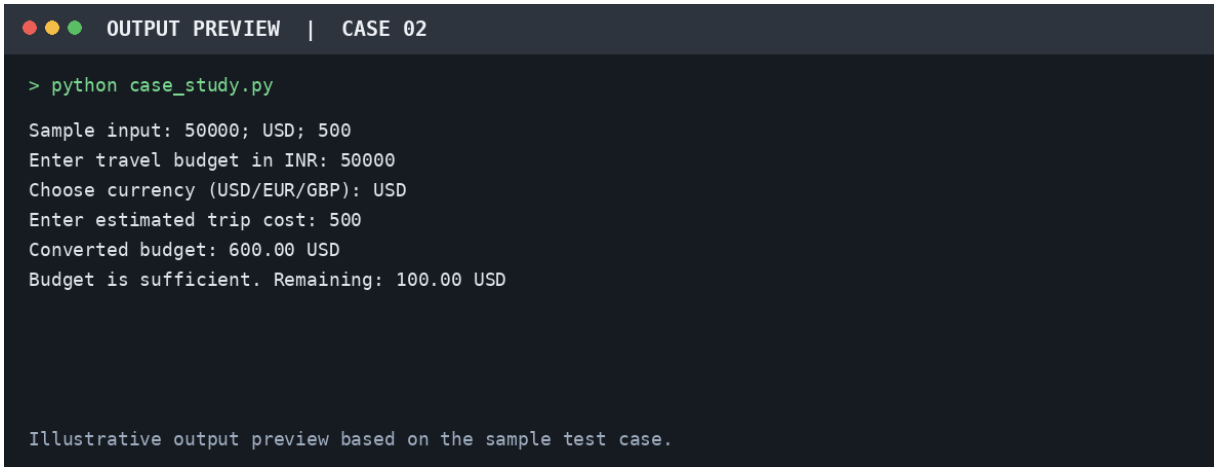
rates = {"USD": 0.012, "EUR": 0.011, "GBP": 0.0095}

if budget_inr < 0 or trip_cost < 0 or currency not in rates:
    print("Invalid input or unsupported currency")
else:
    converted = budget_inr * rates[currency]
    difference = converted - trip_cost
    print(f"Converted budget: {converted:.2f} {currency}")
    if difference >= 0:
        print(f"Budget is sufficient. Remaining: {difference:.2f} {currency}")
    else:
        print(f"Budget shortfall: {abs(difference):.2f} {currency}")
```

8. Test Cases

Test Input / Scenario	Expected Output
Budget ■50,000; USD; cost 500	Converted budget: 600.00 USD; Remaining: 100.00 USD
Budget ■30,000; EUR; cost 400	Converted budget: 330.00 EUR; Shortfall: 70.00 EUR
Budget ■10,000; GBP; cost 50	Converted budget: 95.00 GBP; Remaining: 45.00 GBP
Budget -100; USD; cost 10	Invalid input or unsupported currency

9. Output Screenshot / Preview



10. Conclusion

The program converts a travel budget with a selected sample exchange rate and checks affordability. For real travel planning, current exchange rates and bank or card fees should be checked separately.

CASE STUDY 3: Advanced Numeric Pattern Generator

1. Problem Statement

Develop a pattern generator that accepts a row count and prints an increasing numeric triangle. Each row begins at 1 and ends at the row number, demonstrating nested loops and controlled output formatting.

2. Objectives

• Read and validate the requested number of rows.

• Use an outer loop to control rows and an inner loop to print numbers.

• Display a correctly aligned, easy-to-read numeric pattern.

• Demonstrate how nested loops work.

3. Assumptions

• The allowed row count is from 1 to 9 so the pattern remains compact.

• Each row contains the integers from 1 through that row number.

• The program prints one row per line.

4. Algorithm

1. Start and input the row count.

2. If the row count is less than 1 or greater than 9, display a validation message.

3. Otherwise, start the outer loop from row 1 to the requested row count.

4. For each row, start an inner loop from 1 to the current row number.

5. Print each number on the same line with a space after it.

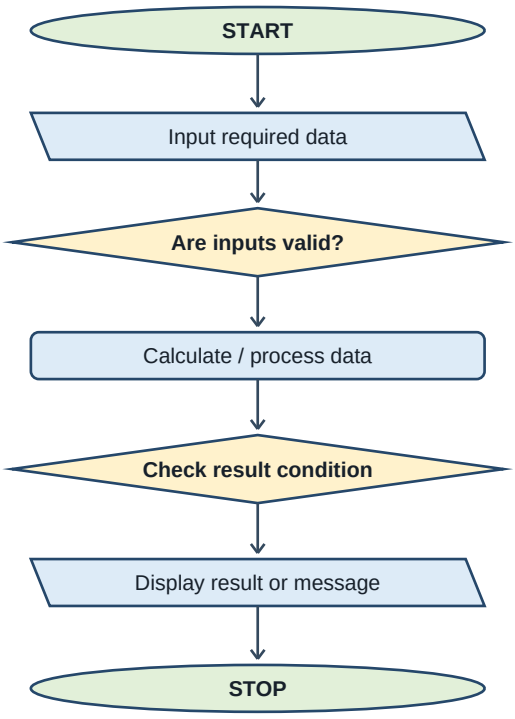
6. Move to a new line after the inner loop finishes.

7. Stop.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 3: IMPLEMENTATION

6. Pseudocode

```
START
INPUT rows
IF rows < 1 OR rows > 9 THEN
    DISPLAY "Enter rows from 1 to 9"
ELSE
    FOR i = 1 TO rows
        FOR j = 1 TO i
            DISPLAY j WITHOUT NEW LINE
        END FOR
        DISPLAY NEW LINE
    END FOR
END IF
STOP
```

7. Python Source Code

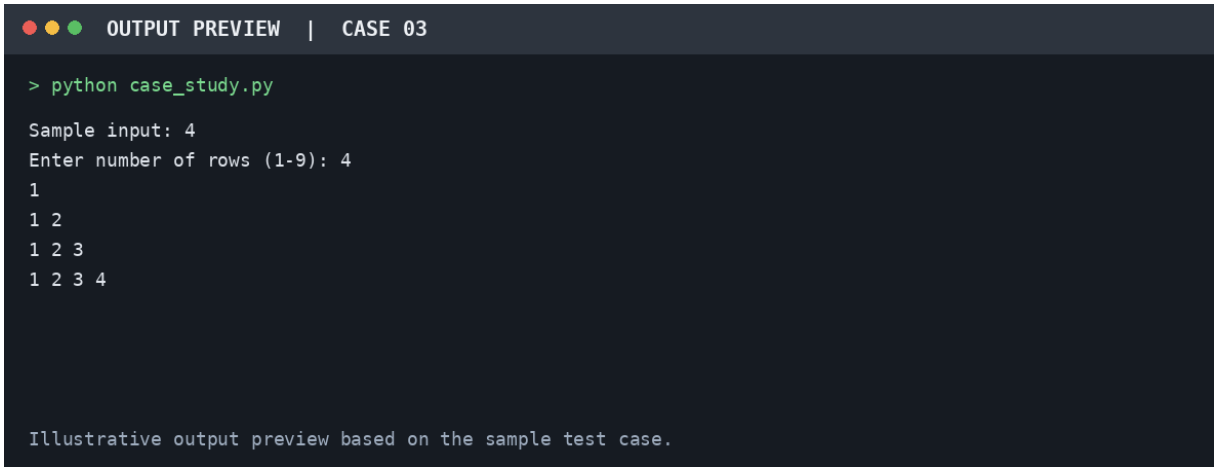
```
rows = int(input("Enter number of rows (1-9): "))

if rows < 1 or rows > 9:
    print("Enter rows from 1 to 9")
else:
    for i in range(1, rows + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()
```

8. Test Cases

Test Input / Scenario	Expected Output
Rows = 4	1\n1 2\n1 2 3\n1 2 3 4
Rows = 1	1
Rows = 0	Enter rows from 1 to 9

9. Output Screenshot / Preview



10. Conclusion

The program uses nested loops to produce a predictable numeric triangle and checks the requested row count before generating the pattern.

CASE STUDY 4: Weekly Electricity Consumption Matrix

1. Problem Statement

Create a program to analyse household electricity consumption for seven days. Store each daily reading, calculate the weekly total and daily average, and identify the day with the highest recorded consumption.

2. Objectives

• Store seven daily readings in a list representing one row of a weekly consumption matrix.

• Validate readings to prevent negative values.

• Calculate the total and average consumption.

• Identify and display the day with the maximum reading.

3. Assumptions

• Exactly seven readings are entered in Monday-to-Sunday order.

• Readings are non-negative numeric values.

• If the maximum value occurs more than once, the first matching day is displayed.

4. Algorithm

1. Start and define the days Monday through Sunday.

2. Use a loop to input one consumption reading for each day.

3. If any reading is negative, display an error.

4. Calculate the sum of all readings and divide by seven to find the average.

5. Find the index of the largest reading.

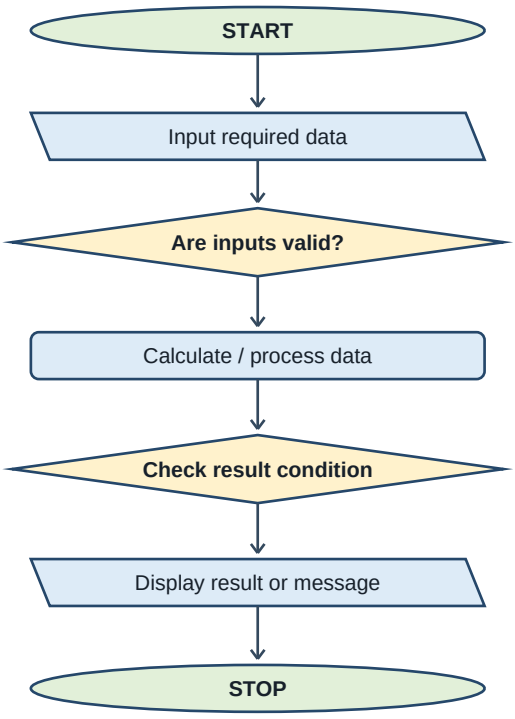
6. Display the weekly total, daily average, and peak-consumption day.

7. Stop.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 4: IMPLEMENTATION

6. Pseudocode

```
START
DAYS = Monday through Sunday
INPUT one reading for each day
IF any reading < 0 THEN
    DISPLAY "Invalid reading"
ELSE
    total = SUM(readings)
    average = total / 7
    peak_index = INDEX OF MAXIMUM reading
    DISPLAY total, average, DAYS[peak_index]
STOP
```

7. Python Source Code

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday",
        "Friday", "Saturday", "Sunday"]
readings = []

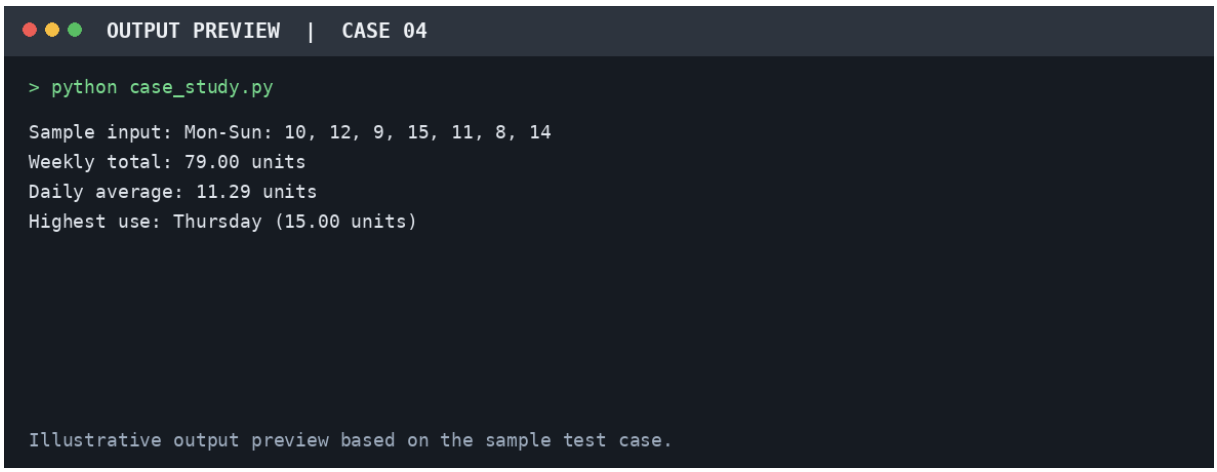
for day in days:
    value = float(input(f"Enter units for {day}: "))
    readings.append(value)

if any(value < 0 for value in readings):
    print("Invalid reading: values cannot be negative")
else:
    total = sum(readings)
    average = total / len(readings)
    peak_index = readings.index(max(readings))
    print(f"Weekly total: {total:.2f} units")
    print(f"Daily average: {average:.2f} units")
    print(f"Highest use: {days[peak_index]} ({readings[peak_index]:.2f} units)")
```

8. Test Cases

Test Input / Scenario	Expected Output
10, 12, 9, 15, 11, 8, 14	Weekly total 79.00; average 11.29; highest use Thursday (15.00 units)
5, 5, 5, 5, 5, 5, 5	Weekly total 35.00; average 5.00; highest use Monday (5.00 units)
10, 12, -1, 15, 11, 8, 14	Invalid reading: values cannot be negative

9. Output Screenshot / Preview



10. Conclusion

The program turns seven daily readings into a useful weekly summary and identifies the peak-consumption day for further energy-saving analysis.

CASE STUDY 5: ATM Transaction Sequence Simulation

1. Problem Statement

Build a menu-driven ATM simulation that lets a user check the balance, deposit money, withdraw money, or exit. The program must update the balance in order and prevent invalid deposits and withdrawals that exceed the available balance.

2. Objectives

• Use a repeating menu to process a sequence of transactions.

• Implement balance enquiry, deposit, withdrawal, and exit options.

• Validate amounts and opening balance.

• Prevent withdrawals that exceed available funds.

3. Assumptions

• The user enters a non-negative opening balance.

• Deposit and withdrawal amounts must be greater than zero.

• This is a classroom simulation only; it does not connect to a bank or handle real money.

4. Algorithm

1. Start and read the opening balance.

2. If the opening balance is negative, display an error and stop.

3. Display the menu inside a loop.

4. For balance enquiry, display the current balance.

5. For deposit, accept a positive amount and add it to the balance.

6. For withdrawal, accept a positive amount; if it exceeds the balance, show insufficient funds; otherwise subtract it.

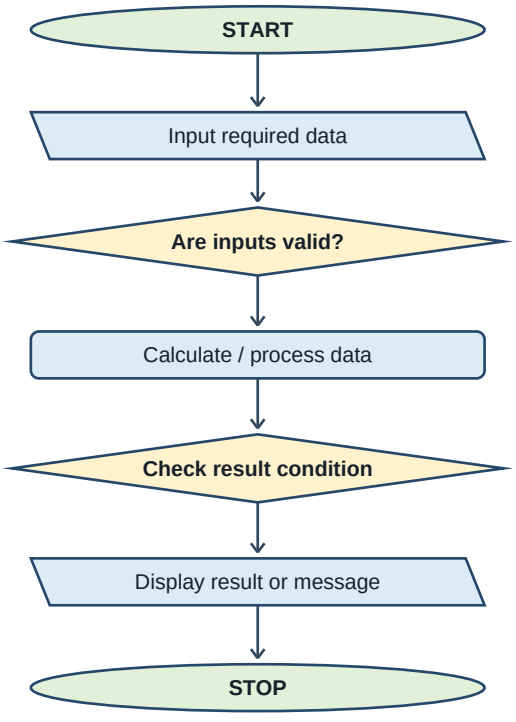
7. For exit, display a closing message and end the loop.

8. For any other choice, display an invalid-choice message.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 5: IMPLEMENTATION

6. Pseudocode

```
START
INPUT balance
IF balance < 0 THEN DISPLAY "Invalid opening balance"
ELSE
    REPEAT
        DISPLAY menu: 1 Balance, 2 Deposit, 3 Withdraw, 4 Exit
        INPUT choice
        IF choice = 1 DISPLAY balance
        ELSE IF choice = 2 INPUT amount; IF amount > 0 ADD amount
        ELSE IF choice = 3 INPUT amount; IF amount > balance DISPLAY insufficient funds ELSE SUBTRACT amount
        ELSE IF choice = 4 DISPLAY goodbye
        ELSE DISPLAY invalid choice
    UNTIL choice = 4
STOP
```

7. Python Source Code

```
balance = float(input("Enter opening balance: "))

if balance < 0:
    print("Invalid opening balance")
else:
    while True:
        print("\n1. Balance enquiry\n2. Deposit\n3. Withdraw\n4. Exit")
        choice = input("Choose an option: ")

        if choice == "1":
            print(f"Available balance: Rs. {balance:.2f}")
        elif choice == "2":
            amount = float(input("Deposit amount: "))
            if amount > 0:
                balance += amount
                print("Deposit successful")
            else:
                print("Amount must be positive")
        elif choice == "3":
            amount = float(input("Withdrawal amount: "))
            if amount <= 0:
                print("Amount must be positive")
            elif amount > balance:
                print("Insufficient balance")
            else:
                balance -= amount
                print("Withdrawal successful")
        elif choice == "4":
            print("Thank you for using the ATM simulation")
            break
        else:
            print("Invalid choice")
```

8. Test Cases

Test Input / Scenario	Expected Output
Opening ■1,000; deposit ■200; withdraw ■100; check balance	Available balance: ■1,100.00
Opening ■500; withdraw ■700	Insufficient balance
Opening balance -1	Invalid opening balance
Deposit amount 0	Amount must be positive

9. Output Screenshot / Preview

OUTPUT PREVIEW | CASE 05

```
> python case_study.py

Sample input: Opening balance 1000; choose 2; deposit 200; choose 3; withdraw 100;
choose 1; choose 4
Deposit successful
Withdrawal successful
Available balance: Rs. 1100.00
Thank you for using the ATM simulation



Illustrative output preview based on the sample test case.
```

10. Conclusion

The simulation demonstrates menu-driven programming, loops, conditions, input validation, and sequential balance updates. It is a learning model rather than a real ATM system.

CASE STUDY 6: Warehouse Inventory Reconciliation

1. Problem Statement

Develop an inventory reconciliation program that compares the quantity recorded in the warehouse system with the physical count. It must identify shortages, excess stock, and exact matches so that discrepancies can be reviewed.

2. Objectives

• Collect the name and recorded and physical quantities for each item.

• Calculate the difference between physical and recorded stock.

• Classify the result as shortage, excess, or matched.

• Reject negative quantities.

3. Assumptions

• The sample processes three items per run.

• Quantities are whole numbers and cannot be negative.

• Difference = physical quantity – recorded quantity.

• The program reports discrepancies but does not automatically change stock records.

4. Algorithm

1. Start.

2. Repeat for three items: read item name, recorded quantity, and physical quantity.

3. If either quantity is negative, display an error for that item and continue to the next item.

4. Calculate the difference between physical and recorded quantities.

5. If the difference is below zero, label it Shortage.

6. If the difference is above zero, label it Excess; otherwise label it Matched.

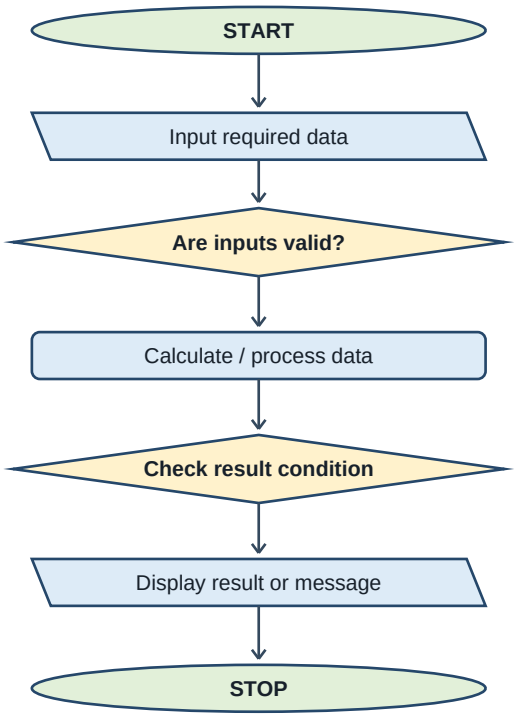
7. Display the item, difference, and status.

8. Stop after all three items have been checked.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 6: IMPLEMENTATION

6. Pseudocode

```
START
FOR each of 3 items
    INPUT name, recorded, physical
    IF recorded < 0 OR physical < 0 THEN
        DISPLAY "Invalid quantity"
    ELSE
        difference = physical - recorded
        IF difference < 0 status = "Shortage"
        ELSE IF difference > 0 status = "Excess"
        ELSE status = "Matched"
        DISPLAY name, difference, status
STOP
```

7. Python Source Code

```
for item_number in range(1, 4):
    name = input(f"Item {item_number} name: ")
    recorded = int(input("Recorded quantity: "))
    physical = int(input("Physical quantity: "))

    if recorded < 0 or physical < 0:
        print("Invalid quantity")
        continue

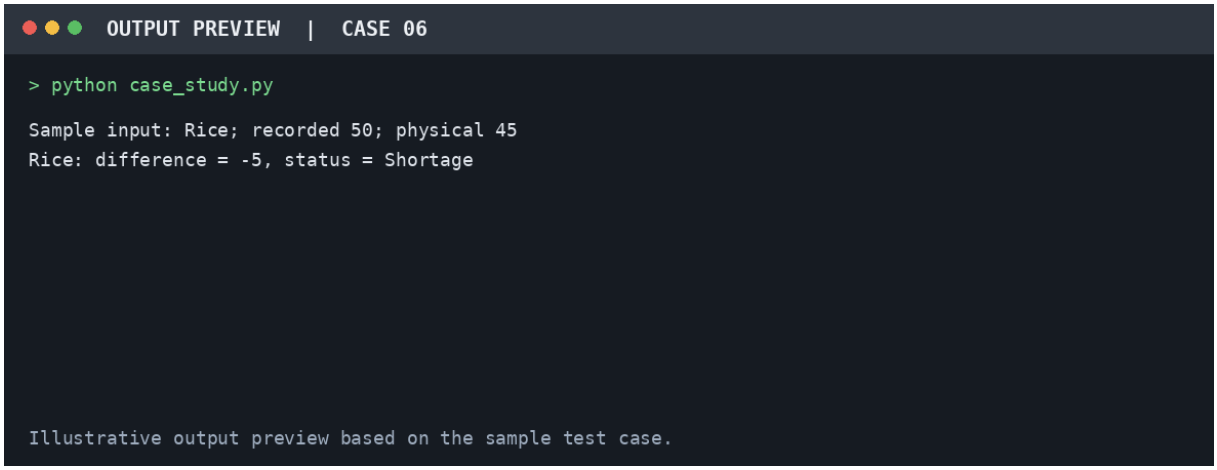
    difference = physical - recorded
    if difference < 0:
        status = "Shortage"
    elif difference > 0:
        status = "Excess"
    else:
        status = "Matched"

    print(f"{name}: difference = {difference}, status = {status}")
```

8. Test Cases

Test Input / Scenario	Expected Output
Rice: recorded 50, physical 45	Rice: difference = -5, status = Shortage
Soap: recorded 20, physical 25	Soap: difference = 5, status = Excess
Oil: recorded 10, physical 10	Oil: difference = 0, status = Matched
Sugar: recorded -2, physical 4	Invalid quantity

9. Output Screenshot / Preview



10. Conclusion

The program provides a simple, repeatable way to reconcile recorded stock against physical counts and flag discrepancies for follow-up.

CASE STUDY 7: Next-Gen Loop Examination Seating Generator

1. Problem Statement

Create a seating-plan generator that assigns sequential roll numbers to seats in classroom rows. It should place only the specified number of students in each row and correctly handle a final row that is not completely filled.

2. Objectives

• Read the first roll number, number of students, and seats per row.

• Use loops to allocate students row by row.

• Display a readable seating plan.

• Validate that all input counts are positive.

3. Assumptions

• Roll numbers are sequential integers.

• First roll number, student count, and seats per row must be greater than zero.

• The final row may contain fewer students than the other rows.

• This sample does not account for alternate seating or special accommodation rules.

4. Algorithm

1. Start and input the first roll number, student count, and seats per row.

2. If any input is not positive, display an error.

3. Set the current roll number to the first roll number and remaining students to the student count.

4. While students remain, create a new row.

5. Fill the row with the smaller of seats-per-row and remaining students.

6. Increase the roll number after every seat assignment.

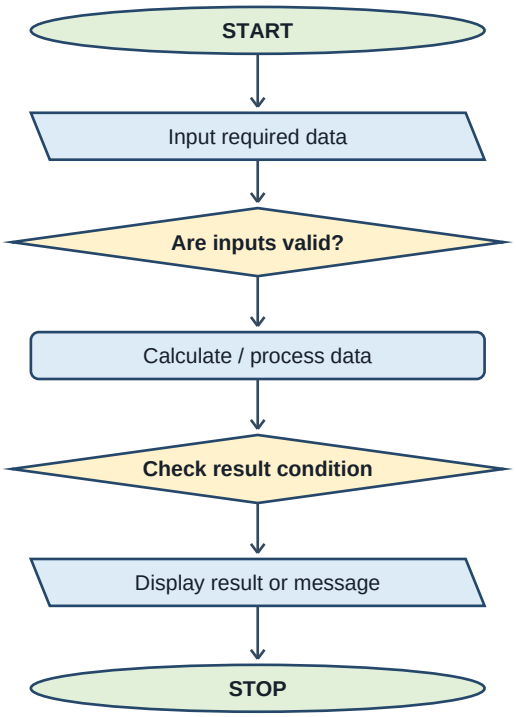
7. Display each row, then continue until no students remain.

8. Stop.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 7: IMPLEMENTATION

6. Pseudocode

```
START
INPUT first_roll, students, seats_per_row
IF any input <= 0 THEN DISPLAY "Invalid input"
ELSE
    current_roll = first_roll
    remaining = students
    row_number = 1
    WHILE remaining > 0
        CREATE empty row
        REPEAT MIN(seats_per_row, remaining) times
            ADD current_roll to row
            current_roll = current_roll + 1
            remaining = remaining - 1
        DISPLAY row_number, row
        row_number = row_number + 1
STOP
```

7. Python Source Code

```
first_roll = int(input("Enter first roll number: "))
students = int(input("Enter number of students: "))
seats_per_row = int(input("Enter seats per row: "))

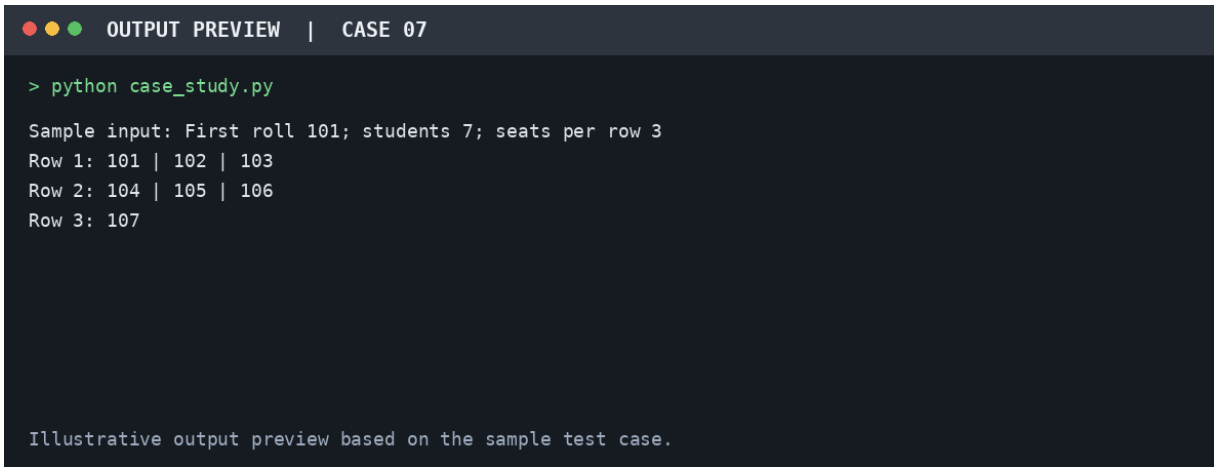
if first_roll <= 0 or students <= 0 or seats_per_row <= 0:
    print("Invalid input")
else:
    current_roll = first_roll
    remaining = students
    row_number = 1

    while remaining > 0:
        row = []
        seats_now = min(seats_per_row, remaining)
        for _ in range(seats_now):
            row.append(current_roll)
            current_roll += 1
            remaining -= 1
        print(f"Row {row_number}: " + " | ".join(map(str, row)))
        row_number += 1
```

8. Test Cases

Test Input / Scenario	Expected Output
First roll 101; 7 students; 3 seats per row	Row 1: 101 | 102 | 103; Row 2: 104 | 105 | 106; Row 3: 107
First roll 1; 2 students; 5 seats per row	Row 1: 1 | 2
Students 0	Invalid input

9. Output Screenshot / Preview



10. Conclusion

The program uses loops to create a sequential seating plan and correctly handles incomplete final rows. Actual examination seating policies should be applied if they differ from sequential allocation.

CASE STUDY 8: University Examination Result Moderation

1. Problem Statement

Write a result-processing program that adds internal and external marks, checks the allowed mark ranges, and applies a clearly stated sample moderation rule to students close to the pass mark. The final result and the original total must both be displayed for transparency.

2. Objectives

• Validate internal and external marks against their maximum values.

• Calculate the original total out of 100.

• Apply the sample moderation rule only to specified near-pass totals.

• Display original total, moderated total, and final result.

3. Assumptions

• Internal marks are out of 25 and external marks are out of 75.

• The pass mark is 40 out of 100.

• For demonstration only, original totals of 38 or 39 are adjusted to 40.

• This is not an official university regulation; the institution's published rules must be used for real results.

4. Algorithm

1. Start and input internal and external marks.

2. If internal marks are outside 0–25 or external marks are outside 0–75, display an error.

3. Add the two components to calculate the original total.

4. If the total is 38 or 39, set the sample moderated total to 40; otherwise keep the original total.

5. If the final total is at least 40, set the result to PASS; otherwise set it to FAIL.

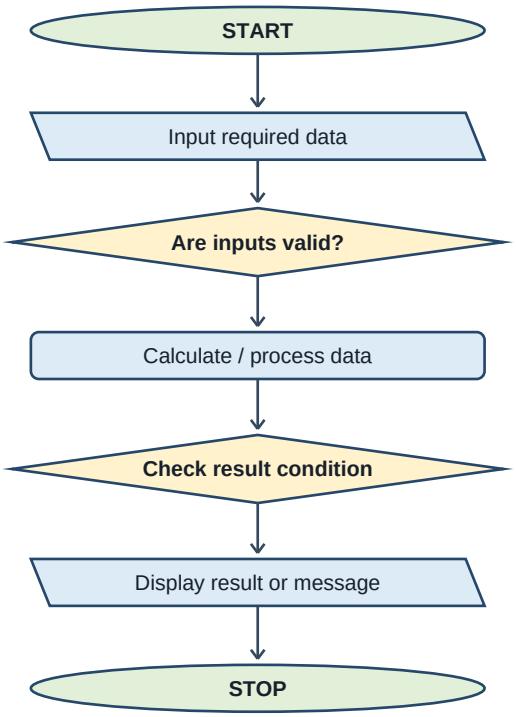
6. Display original total, final marks, and result.

7. Stop.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 8: IMPLEMENTATION

6. Pseudocode

```
START
INPUT internal, external
IF internal < 0 OR internal > 25 OR external < 0 OR external > 75 THEN
    DISPLAY "Invalid marks"
ELSE
    total = internal + external
    IF total = 38 OR total = 39 THEN final = 40
    ELSE final = total
    IF final >= 40 THEN result = PASS ELSE result = FAIL
    DISPLAY total, final, result
STOP
```

7. Python Source Code

```
internal = int(input("Enter internal marks (0-25): "))
external = int(input("Enter external marks (0-75): "))

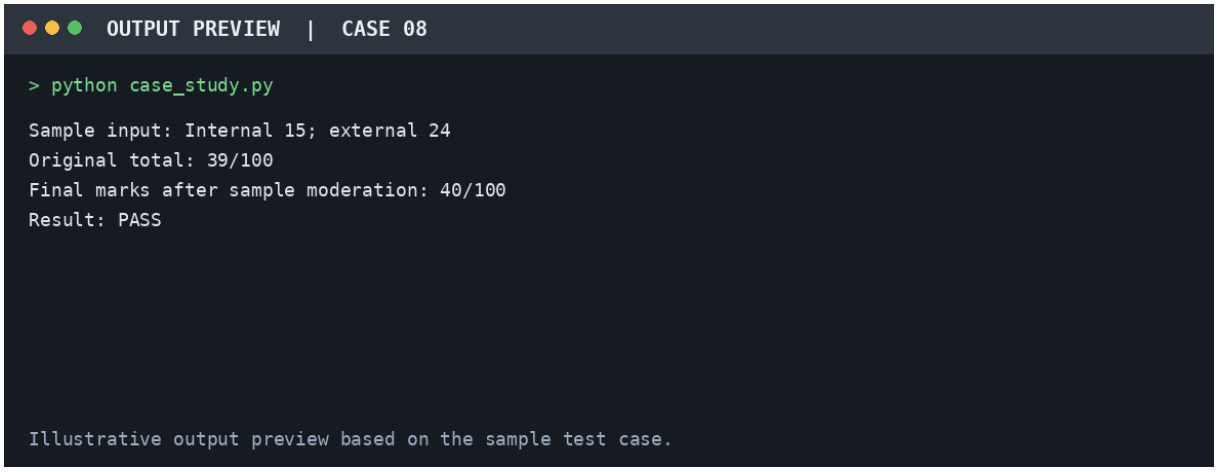
if not (0 <= internal <= 25 and 0 <= external <= 75):
    print("Invalid marks")
else:
    total = internal + external
    final_marks = 40 if total in (38, 39) else total
    result = "PASS" if final_marks >= 40 else "FAIL"

    print(f"Original total: {total}/100")
    print(f"Final marks after sample moderation: {final_marks}/100")
    print(f"Result: {result}")
```

8. Test Cases

Test Input / Scenario	Expected Output
Internal 15; external 24	Original total 39/100; final marks 40/100; PASS
Internal 10; external 20	Original total 30/100; final marks 30/100; FAIL
Internal 26; external 50	Invalid marks
Internal 25; external 75	Original total 100/100; final marks 100/100; PASS

9. Output Screenshot / Preview



10. Conclusion

The program validates component marks, calculates totals, and illustrates how a moderation rule can be represented in code. Actual examination results must follow the university's approved regulations.

CASE STUDY 9: Smart Public Transport Fare Reconciliation

1. Problem Statement

Develop a program that calculates a sample public-transport fare from the passenger's travel distance and compares the fare with the payment received. The program should display the change due or the remaining amount payable.

2. Objectives

• Read and validate the distance and amount paid.

• Calculate the fare using a base fare and a per-kilometre rate.

• Compare payment received with the fare due.

• Display the change or unpaid balance.

3. Assumptions

• Sample fare = ■10 base fare + ■2 per kilometre.

• Distance must be greater than zero and payment cannot be negative.

• The rates are illustrative only and are not an official transport tariff.

4. Algorithm

1. Start and input distance and payment received.

2. If distance is not positive or payment is negative, display an error.

3. Calculate fare as 10 + distance × 2.

4. If payment is at least the fare, calculate and display change.

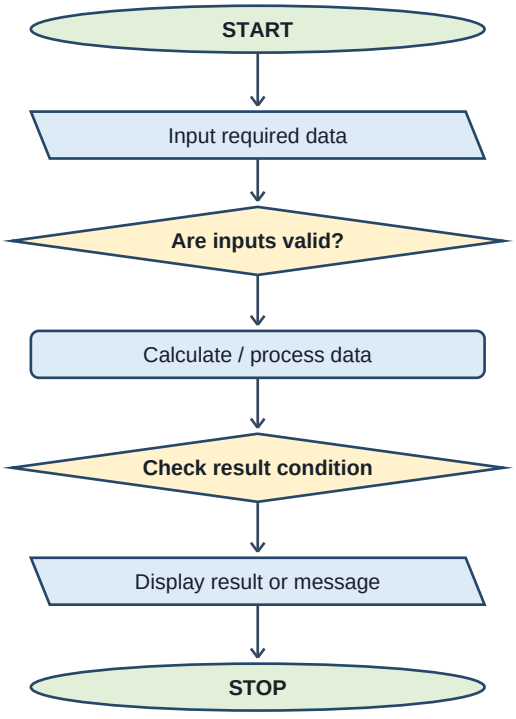
5. Otherwise, calculate and display the amount still due.

6. Stop.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 9: IMPLEMENTATION

6. Pseudocode

```
START
INPUT distance, paid
IF distance <= 0 OR paid < 0 THEN DISPLAY "Invalid input"
ELSE
    fare = 10 + distance * 2
    DISPLAY fare
    IF paid >= fare THEN DISPLAY change = paid - fare
    ELSE DISPLAY amount due = fare - paid
STOP
```

7. Python Source Code

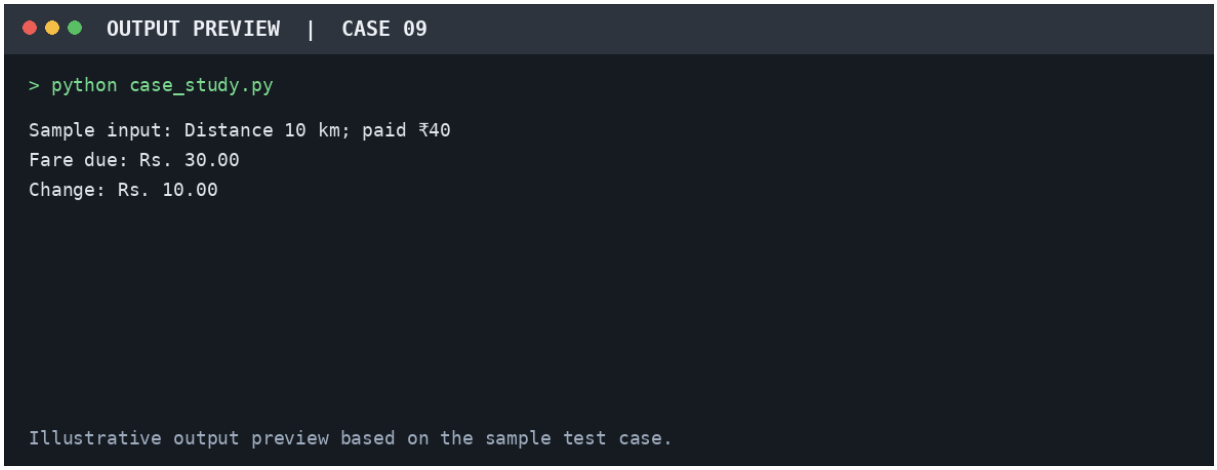
```
distance = float(input("Enter distance in kilometres: "))
paid = float(input("Enter amount paid (Rs.): "))

if distance <= 0 or paid < 0:
    print("Invalid input")
else:
    fare = 10 + distance * 2
    print(f"Fare due: Rs. {fare:.2f}")
    if paid >= fare:
        print(f"Change: Rs. {paid - fare:.2f}")
    else:
        print(f"Amount still due: Rs. {fare - paid:.2f}")
```

8. Test Cases

Test Input / Scenario	Expected Output
Distance 10 km; paid ■40	Fare due ■30.00; Change ■10.00
Distance 5 km; paid ■15	Fare due ■20.00; Amount still due ■5.00
Distance 0 km; paid ■20	Invalid input
Distance 4 km; paid ■0	Fare due ■18.00; Amount still due ■18.00

9. Output Screenshot / Preview



10. Conclusion

The program calculates a sample fare and reconciles it with the amount paid, showing either change or an outstanding amount.

CASE STUDY 10: Water Distribution Fairness Analysis

1. Problem Statement

Create a program to analyse household water distribution using a list of household consumption values. It should calculate the total and average consumption, compare each household's use with the average, and flag households whose consumption is substantially higher than the average for further review.

2. Objectives

• Collect water-use values for a group of households.

• Calculate total and average consumption.

• Compare each household's consumption with the group average.

• Identify households using more than 150% of the average without assuming that high use automatically means unfairness.

3. Assumptions

• The sample uses five households; the number can be changed in the program.

• Consumption values must be non-negative.

• A household is flagged for review when its use is greater than 1.5 times the average.

• A flag is only a prompt for review; household size, medical needs, leaks, and local allocation rules may explain differences.

4. Algorithm

1. Start and input water consumption for five households.

2. If any reading is negative, display an error.

3. Calculate the total consumption and average consumption.

4. Calculate the review threshold as 1.5 times the average.

5. For each household, display its consumption and compare it with the threshold.

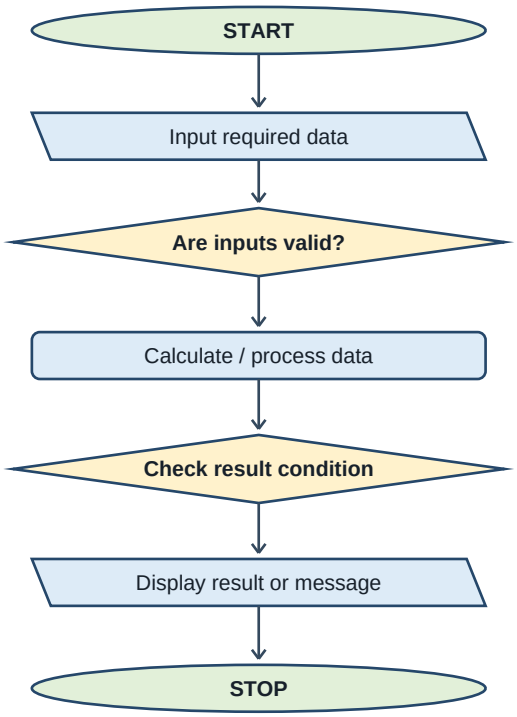
6. If consumption is above the threshold, label it 'Review high use'; otherwise label it 'Within threshold'.

7. Display the total, average, and threshold, then stop.

5. Flowchart

Follow the flow from START to STOP. The diamond symbols represent validation or result decisions.

Yes/No decisions are represented; follow the code for exact branches.



Oval: start/end | Parallelogram: input/output | Rectangle: process | Diamond: decision

CASE STUDY 10: IMPLEMENTATION

6. Pseudocode

```
START
INPUT consumption for 5 households
IF any value < 0 THEN DISPLAY "Invalid consumption"
ELSE
    total = SUM(consumption)
    average = total / 5
    threshold = average * 1.5
    DISPLAY total, average, threshold
    FOR each household
        IF use > threshold THEN status = "Review high use"
        ELSE status = "Within threshold"
        DISPLAY household, use, status
STOP
```

7. Python Source Code

```
consumption = []

for household in range(1, 6):
    value = float(input(f"Enter water use for household {household} (kL): "))
    consumption.append(value)

if any(value < 0 for value in consumption):
    print("Invalid consumption: values cannot be negative")
else:
    total = sum(consumption)
    average = total / len(consumption)
    threshold = average * 1.5

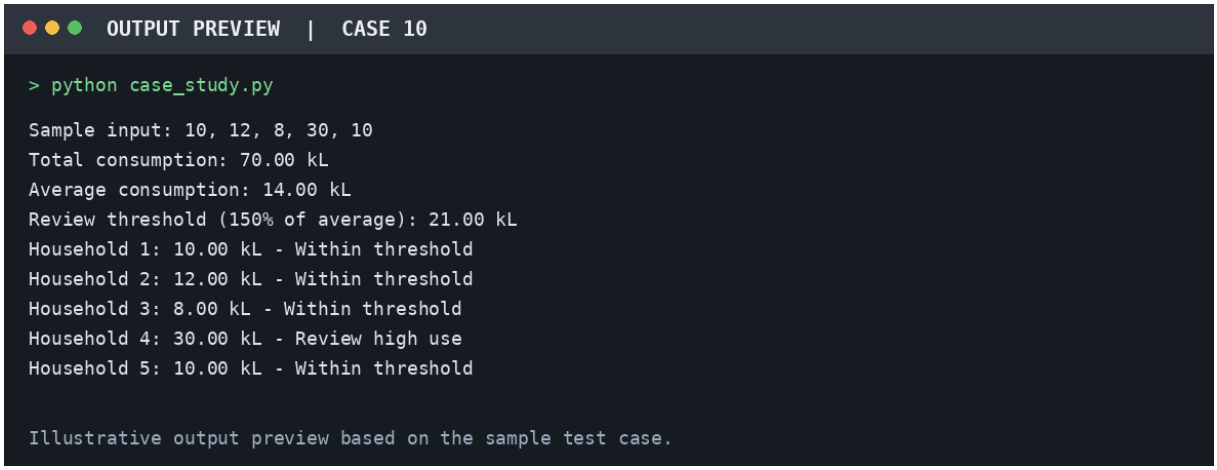
    print(f"Total consumption: {total:.2f} kL")
    print(f"Average consumption: {average:.2f} kL")
    print(f"Review threshold (150% of average): {threshold:.2f} kL")

    for index, value in enumerate(consumption, start=1):
        if value > threshold:
            status = "Review high use"
        else:
            status = "Within threshold"
        print(f"Household {index}: {value:.2f} kL - {status}")
```

8. Test Cases

Test Input / Scenario	Expected Output
Households: 10, 12, 8, 30, 10	Total 70.00 kL; average 14.00 kL; threshold 21.00 kL; Household 4 flagged for review
Households: 5, 5, 5, 5, 5	Total 25.00 kL; average 5.00 kL; threshold 7.50 kL; all within threshold
One value is -3	Invalid consumption: values cannot be negative

9. Output Screenshot / Preview



10. Conclusion

The program provides a simple comparison of household water consumption and flags unusually high values for investigation. Fairness decisions should also consider household circumstances and official distribution rules.